

Генетичний алгоритм еволюційного моделювання

Покрокове пояснення алгоритму, реалізованого у застосунку *DeliveryIQ* (модуль *vrp_solver.py*)

Документ описує, як реалізовано генетичний алгоритм (ГА) еволюційного моделювання для розв'язання задачі маршрутизації транспортних засобів з обмеженнями вантажопідйомності та часовими вікнами (CVRPTW). Кожен крок алгоритму викладено послідовно: від кодування хромосоми до завершення еволюційного циклу та формування фінальних маршрутів.

Реалізація ґрунтується на класичній схемі ГА Голланда (1975) із адаптаціями для VRP: декодером Split (Prins, 2004), оператором впорядкованого схрещування Order Crossover (Oliver, Smith, Holland, 1987) та елітизмом для збереження найкращих розв'язків.

1. Постановка задачі

Задано n точок доставки, депо D та парк K транспортних засобів (ТЗ) з різними режимами руху (drive / bike / walk) та вантажопідйомностями Q_θ . Кожна точка має вагу w_i (кг), часове вікно $[tw_open_i, tw_close_i]$ та час обслуговування s_i .

Потрібно мінімізувати цільову функцію:

$$F = A \cdot T_{total} + B \cdot K_{active} + P \cdot |U|$$

де:

- T_{total} — сумарний час подорожі всіх ТЗ;
- K_{active} — кількість використаних ТЗ;
- U — множина незамаршрутизованих клієнтів;
- $A = 1.0$, $B = 0.0$, $P = 10^6$ — вагові коефіцієнти (штраф $P \gg A, B$ змушує ГА уникати неповних розв'язків).

Обмеження:

- **Capacity:** $\sum w_i$ по маршруту $\leq Q_\theta$;
- **Time-window:** прибуття у точку $i \leq tw_close_i$;
- **Round-trip:** кожен ТЗ стартує з депо й повертається у депо до tw_close_D ;
- **Reachability:** кожне ребро маршруту прохідне для режиму ТЗ (вартість $< PENALTY = 10^9$).

2. Двофазна архітектура

Алгоритм у *vrp_solver.py* побудовано як двофазну схему. Це принципове проєктне рішення: воно розбиває складну багатомодальну задачу CVRPTW на серію незалежних одно-модальних підзадач, кожну з яких ГА розв'язує ефективніше.

Фаза	Функція	Призначення
1	<code>_assign_stops_to_modes()</code>	Розподіл точок між пулами режимів (drive/bike/walk) за критерієм досяжності
2	<code>_genetic_algorithm()</code>	Окремий запуск ГА у кожному пулі — пошук розподілу точок між ТЗ і порядком їх об'їзду

Чому двофазна схема: ТЗ режиму walk не може використовувати ребра, прохідні лише для bike, тож зведення задачі до одного ГА над усіма точками потребувало б складніших обмежень у декодері. Розподіл за режимами заздалегідь спрощує операторів ГА і робить декодер чистішим.

3. Кодування розв'язку — хромосома

Кожен **індивід** популяції кодується як **перестановка** індексів клієнтів пулу — так званий *гігантський тур* (giant tour). Розділювачів між маршрутами різних ТЗ у хромосомі немає — їх вставляє декодер під час оцінки.

Приклад

Нехай у пулі режиму *drive* 6 точок з індексами 0...5. Хромосома може виглядати так:

```
chromo = [3, 0, 5, 1, 4, 2]
```

Це означає: декодер спочатку спробує покласти у поточний маршрут точку 3, потім 0, потім 5 і т.д. Якщо у певний момент обмеження порушується — поточний маршрут закривається, береться наступний ТЗ із флоту.

Властивості кодування

- **Універсальна допустимість:** будь-яка перестановка валідна як хромосома — допустимість маршрутів забезпечує декодер.
- **Без надмірності:** один розв'язок $\leftrightarrow$ одна хромосома (ігноруючи циклічні зсуви всередині сегментів).
- **Сумісність з операторами TSP:** класичні ОХ-кросовер та swap-мутація працюють напівпрямую без модифікацій.

4. Ініціалізація популяції

Розмір популяції — **POP_SIZE = 50** хромосом. Популяція формується гібридно: один **теплий старт** + випадкове наповнення.

4.1. Жадібний сід (Nearest-Neighbour)

Функція **_nearest_neighbour_seed()** будує одну хромосому, обираючи на кожному кроці найближчу за часом подорожі ще не відвідану точку:

```
current = depot_node
while unvisited:
    next = argmin_{c in unvisited} matrix[current, stop_node[c]]
    chromo.append(next)
    current = stop_node[next]
```

4.2. Випадкові перестановки

Решта (POP_SIZE - 1) хромосом — випадкові перестановки **list(range(n))**, перемішані через **rng.shuffle()**. Це забезпечує початкову різноманітність.

Чому саме гібридний старт: чисто випадкова популяція потребує багато поколінь, щоб дістатися до якісних розв'язків. Один NN-сід одразу задає верхню межу фітнесу — ГА може лише поліпшувати її далі. Чисто жадібна ініціалізація, навпаки, обмежить дослідження простору розв'язків і ризикує застрягти у локальному оптимумі.

5. Функція пристосованості (фітнес)

Кожна хромосома оцінюється функцією `_evaluate()`, яка повертає трійку (**fitness**, **routes**, **unassigned**). Чим менший фітнес — тим кращий індивід (мінімізаційна постановка).

5.1. Декодер Split

Перш ніж порахувати фітнес, потрібно перетворити хромосому (перестановку) на конкретний розклад маршрутів — це робить `_decode_chromosome()` (метод Split, Prins 2004). Алгоритм проходить хромосому зліва направо й жадібно наповнює поточний ТЗ клієнтами:

1. Беремо першого ТЗ із флоту: **vehicle** = **vehicles[0]**, ініціалізуємо порожній **current_route** = [].
2. Для кожного гену **cust** з хромосоми перевіряємо: чи буде маршрут **current_route** + **[cust]** допустимим для поточного ТЗ?
3. **Якщо так** — додаємо клієнта до поточного маршруту й переходимо до наступного гену.
4. **Якщо ні** — закриваємо поточний маршрут (вкладаємо у список *routes*), беремо наступний ТЗ і повторюємо перевірку для цього самого гену.
5. Якщо клієнт не вмістився у жодного з решти ТЗ — він додається до **unassigned**.

5.2. Функція допустимості маршруту

Функція `route_feasible(route, vehicle)` перевіряє три обмеження одночасно (псевдокод):

```
def route_feasible(route, vehicle):
    # Capacity
    if sum(weight[i] for i in route) > vehicle.capacity_kg:
        return False

    # Time-window + reachability simulation
    prev = depot; prev_begin = depot.tw_open
    for cust in route:
        tt = matrix[prev -> cust]
        if tt >= PENALTY: return False # impassable
        arrival = prev_begin + service[prev] + tt
        if arrival > tw_close[cust]: return False # late
        prev_begin = max(arrival, tw_open[cust]) # wait if early
        prev = cust

    # Round-trip back to depot before its tw_close
    tt_back = matrix[prev -> depot]
    if tt_back >= PENALTY: return False
    if prev_begin + service[prev] + tt_back > depot.tw_close:
        return False
    return True
```

5.3. Обчислення фітнесу

Після декодування фітнес обчислюється за формулою:

$$F = A \cdot \sum T_r + B \cdot K + P \cdot |U|$$

де T_r — час маршруту r (час $\text{depot} \rightarrow c_1 \rightarrow \dots \rightarrow c_k \rightarrow \text{depot}$ з матриці), K — кількість активних маршрутів, $|U|$ — кількість невпихнутих клієнтів.

Штраф $P \cdot |U|$ змушує ГА спершу маршрутизувати ВСІХ клієнтів. Лише після цього оптимізатор починає скорочувати T_{total} . Це властивість лексикографічної оптимізації.

6. Генетичні оператори

6.1. Селекція — турнірний відбір

Турнір розміру **TOURNAMENT_SIZE = 3** працює так:

1. Випадково обираються 3 індекси з популяції (без повторень).
2. Серед них обирається індивід з **найменшим** фітнесом.
3. Цей індивід стає батьком для подальшого схрещування.

Реалізація — **_tournament_select()**:

```
def tournament_select(pop, fitnesses, k=3):
    candidates = rng.sample(range(len(pop)), k)
    best = min(candidates, key=lambda i: fitnesses[i])
    return pop[best]
```

Чому турнір, а не пропорційна (рулеткова) селекція: турнір не залежить від абсолютних значень фітнесу й коректно працює, навіть коли частина хромосом має штраф $P \cdot |U|$ □ T_{total} Рулетка ж в такому випадку зосередила б усю ймовірність на одному-двох індивідах.

6.2. Кросовер — Order Crossover (OX)

OX — оператор для перестановок, що зберігає *відносний* порядок генів. На вхід — два батьки p_1 , p_2 ; на вихід — одна дитина (**CROSSOVER_RATE = 0.85** — імовірність застосування; інакше дитина = копія p_1).

Покрокова робота OX

1. Випадково обираються дві позиції $i < j$ у $[0, n-1]$.
2. Сегмент $p_1[i..j]$ копіюється у дитину на ті ж позиції (це 'збережене ядро' від першого батька).
3. Інші позиції дитини заповнюються генами з p_2 у порядку появи, починаючи з позиції $j+1$ (з циклічним обходом). Гени, вже присутні у ядрі, пропускаються.

Приклад OX ($n = 6, i = 2, j = 4$)

```
p1 = [3, 0, |5, 1, 4|, 2] ← копіюємо сегмент [5,1,4]
p2 = [2, 4, 1, 5, 3, 0]
```

Кроки:

- 1) ядро дитини: $[_, _, 5, 1, 4, _]$
- 2) сканування p_2 з позиції $j+1=5$:
 $[0, 2, 4 \text{ (skip)}, 1 \text{ (skip)}, 5 \text{ (skip)}, 3]$
→ корисні в порядку: 0, 2, 3
- 3) заповнення з позиції 5 (циклічно): $\text{pos } 5 = 0$,
 $\text{pos } 0 = 2$, $\text{pos } 1 = 3$.

```
child = [2, 3, 5, 1, 4, 0]
```

6.3. Мутація — swap

З імовірністю **MUTATION_RATE = 0.15** у дитині міняються місцями значення двох випадкових позицій. Це найпростіша операція, що зберігає інваріант перестановки.

```
i, j = rng.sample(range(n), 2)
chromo[i], chromo[j] = chromo[j], chromo[i]
```

Чому swap, а не складніші мутації (inversion, scramble): декодер Split дуже чутливий до позиції генів — навіть мала зміна (1 swap) часто дає інший розподіл клієнтів між ТЗ. Сильніші мутації руйнують структуру маршрутів і фітнес різко погіршується.

6.4. Елітизм

Перед формуванням нового покоління **ELITE_SIZE = 2** найкращих хромосом з поточної популяції копіюються у нову без змін. Це гарантує, що найкращий знайдений фітнес **не зростає** від покоління до покоління.

```
order = sorted(range(POP_SIZE), key=lambda i: fitnesses[i])
new_population = [population[i][:] for i in order[:ELITE_SIZE]]
```

7. Основний еволюційний цикл

Функція `_genetic_algorithm()` об'єднує всі попередні компоненти у головний цикл:

```
# 1) Initial population (1 NN seed + random fillers)
population = [nearest_neighbour_seed(...)] + [shuffle(range(n)) ...]
evaluations = [_evaluate(c, ...) for c in population]
fitnesses = [e[0] for e in evaluations]
best_eval = min(evaluations, key=lambda e: e[0])

# 2) Evolutionary loop
for generation in range(N_GENERATIONS):
    # 2a) Elitism – keep the top E
    order = sorted(range(POP_SIZE), key=lambda i: fitnesses[i])
    new_pop = [population[i][:] for i in order[:ELITE_SIZE]]

    # 2b) Fill the rest of the new population
    while len(new_pop) < POP_SIZE:
        p1 = tournament_select(population, fitnesses)
        p2 = tournament_select(population, fitnesses)
        child = order_crossover(p1, p2) if rand < CROSSOVER_RATE
        else p1[:]
        if rand < MUTATION_RATE:
            child = swap_mutation(child)
        new_pop.append(child)

    # 2c) Replace and re-evaluate
    population = new_pop
    evaluations = [_evaluate(c, ...) for c in population]
    fitnesses = [e[0] for e in evaluations]

    # 2d) Track best-so-far
    gen_best = min(evaluations, key=lambda e: e[0])
    if gen_best[0] < best_eval[0]:
        best_eval = gen_best

# 3) Decode the best chromosome to final routes
_, best_routes, unassigned = best_eval
return best_routes
```

7.1. Покрокове резюме одного покоління

1. **Сортування:** популяція сортується за фітнесом за зростанням.
2. **Елітизм:** топ-2 індивіди копіюються у нову популяцію.
3. **Селекція:** турнір розміру 3 обирає двох батьків.
4. **Кросовер:** з імовірністю 0.85 — ОХ; інакше дитина = копія першого батька.
5. **Мутація:** з імовірністю 0.15 — swap двох позицій.
6. **Додавання дитини** до нової популяції; повтор кроків 3–6, поки нова популяція не досягне `POP_SIZE = 50`.
7. **Заміна:** стара популяція повністю замінюється на нову (generational replacement).
8. **Переоцінка:** для всіх 50 нових хромосом викликається `_evaluate()` — обчислюється фітнес і декодуються маршрути.
9. **Оновлення best-so-far:** якщо знайдено новий найкращий індивід — він зберігається.

8. Завершення та формування результату

8.1. Критерій зупинки

У реалізації використано **фіксовану кількість поколінь** **N_GENERATIONS = 150**. Альтернативні критерії (стагнація найкращого фітнесу, ліміт часу) не застосовано — фіксована кількість поколінь забезпечує детерміновану тривалість запуску, що зручно для UI Streamlit.

8.2. Декодування найкращої хромосоми

Після завершення циклу зберігається **best_eval** — трійка (фітнес, маршрути, unassigned) для найкращого індивіда за всю історію. **best_routes** — це список пар (**Vehicle**, [індекси клієнтів у порядку відвідання]).

8.3. Реконструкція повного маршруту

Функція **_build_ga_routes()** для кожного маршруту:

1. Будує замкнений тур по OSM-вузлах: **tsp_route = [depot] + [stop_node[i] for i in indices] + [depot]**.
2. Викликає **reconstruct_full_route(G, tsp_route)** — для кожної пари сусідніх вузлів запускає Дейкстру і склеює сегменти, утворюючи повну послідовність OSM-вузлів для рендеру полілінії.
3. Підсумовує час за матрицею (для метрик) і довжину за атрибутом **length** ребер графа (для відображення в UI).
4. Повертає об'єкт **VehicleRoute** з полями: T3, упорядкований список зупинок, tsp_route, full_route, total_time_s, total_dist_m.

8.4. Обробка нерозподілених клієнтів

Якщо **unassigned** непорожній — це означає, що навіть найкращий індивід не зміг розмістити всі точки (через перевантаження флоту або жорсткі часові вікна). Logger пише попередження, а у UI з'являється банер з переліком пропущених адрес.

9. Гіперпараметри

Усі параметри визначено у верхній частині **vrp_solver.py** (рядки ~80–92). Значення підібрані емпірично та відповідають рекомендаціям §2.7 дипломної роботи.

Параметр	Значення	Призначення
POP_SIZE	50	Кількість хромосом у популяції.
N_GENERATIONS	150	Кількість ітерацій еволюційного циклу.
CROSSOVER_RATE	0.85	Імовірність застосування ОХ-кросоверу до пари батьків.
MUTATION_RATE	0.15	Імовірність swap-мутації для дитини.
ELITE_SIZE	2	Скільки найкращих індивідів переходить у наступне покоління без змін.
TOURNAMENT_SIZE	3	Розмір вибірки для турнірної селекції.

UNROUTED_PENALTY	10 ⁶	Штраф за кожного незамаршрутизованого клієнта (P у формулі фітнесу).
GA_SEED	42	Зерно RNG; None — недетерміновані запуски.
OBJ_A	1.0	Вага сумарного часу T_total у фітнесі.
OBJ_B	0.0	Вага кількості активних ТЗ К (за замовчуванням вимкнена).

GA_SEED = 42 робить запуски детермінованими: для тих самих вхідних даних ГА завжди повертає той самий розподіл маршрутів. Це критично для відтворюваності у дипломному дослідженні та для стабільності UI.

10. Покроковий приклад роботи

Розгляньмо умовний сценарій: 5 точок доставки, флот із двох ТЗ режиму *drive* з вантажопідйомностями 10 кг та 8 кг.

Вхідні дані

№	Адреса	Вага (кг)	Часове вікно
0	вул. А, 12	3.5	09:00–12:00
1	вул. Б, 7	5.0	10:00–14:00
2	пр. В, 33	2.0	09:00–18:00
3	вул. Г, 4	4.5	11:00–15:00
4	вул. Д, 21	3.0	09:00–18:00

Крок 1. Ініціалізація

Початкова популяція (50 хромосом):

- NN-сід (нагадаємо, перший знайдений жадібно): **[2, 4, 0, 1, 3]**
- 49 випадкових перестановок: **[3, 1, 4, 0, 2], [0, 2, 1, 3, 4], ...**

Крок 2. Перша оцінка

Декодер Split проходить кожну хромосому. Наприклад, для NN-сиду:

chromo = [2, 4, 0, 1, 3]

ТЗ #1 (Q=10): додає 2 (w=2.0, ост.=8.0), 4 (w=3.0, ост.=5.0), 0 (w=3.5, ост.=1.5). Спроба додати 1 (w=5.0) — ні, перевищує місткість.

→ маршрут №1: depot → 2 → 4 → 0 → depot.

ТЗ #2 (Q=8): додає 1 (w=5.0), 3 (w=4.5) — сумарно 9.5, перевищення. Беремо лише 1, потім 3 не вміщається. Якщо ТЗ більше немає — 3 потрапляє у *unassigned*.

Фітнес = $T_{total} + 10^6 \cdot 1 \approx \text{велике число}$

Крок 3. Селекція + кросовер

Турнір обирає двох батьків (припустимо):

p1 = **[2, 4, 0, 1, 3]** (NN-сід, фітнес найнижчий)

p2 = **[3, 1, 4, 0, 2]**

ОХ з i=1, j=3 → ядро p1[1..3] = [4, 0, 1]

Заповнення з p2: на позиціях 4, 0 ставимо **3, 2**

child = **[2, 4, 0, 1, 3]**

Крок 4. Мутація

3 імовірністю 0.15 застосовується swap. Припустимо, обрано позиції 0 та 4:

child = [2, 4, 0, 1, 3] → **[3, 4, 0, 1, 2]**

Тепер декодер може знайти інший розподіл — наприклад, ТЗ #1 везе [3, 4, 0] а ТЗ #2 везе [1, 2]. Якщо так — точка 3 розподілена і штраф Р зникне.

Крок 5. Покоління 2...150

Цикл повторюється:

З кожним поколінням еліта 2 індивідів зберігається без змін, решта 48 формуються через селекцію → кросовер → мутацію. Найкращий фітнес монотонно не зростає (елітизм)

гарантує). На пізніх поколіннях популяція збігається — більшість хромосом схожі, мутація приносить дрібні поліпшення.

Крок 6. Фінал

Після 150 поколінь:

Беремо `best_eval` (зберігся з усієї історії) → декодуємо у **`best_routes`** = [(ТЗ #1, [2, 4, 0]), (ТЗ #2, [1, 3])].

Кожен маршрут реконструюється у повну послідовність OSM-вузлів через Дейкстру. UI Streamlit рендерить дві кольорові полілінії на карті Folium.

11. Обґрунтування проєктних рішень

11.1. Чому ГА, а не точний метод

CVRPTW — NP-важка задача. Точні методи (branch-and-cut, column generation) масштабуються лише до ~100 клієнтів і потребують комерційних розв'язувачів (CPLEX, Gurobi). ГА — метаевристика, що дає якісне (хоч і не гарантовано оптимальне) рішення за прийнятний час, працює без зовнішніх залежностей і легко розширюється.

11.2. Чому permutation encoding замість explicit assignment

Альтернатива — кодувати кожну хромосому як [ТЗ, точка, позиція]-трійки. Проте таке кодування створює *надмірність* (декілька кодів описують один розв'язок) і вимагає складних операторів кросоверу, що зберігають допустимість. Permutation encoding простіший і використовує перевірені оператори TSP.

11.3. Чому Split-декодер

Декодер Split (Prins, 2004) — стандарт для VRP-ГА. Його ключові переваги:

- **Конструктивність:** завжди повертає допустимий розклад (або позначає клієнтів як unassigned).
- **Лінійна складність:** $O(n \cdot K)$ по одному проходу хромосоми.
- **Сумісність з обмеженнями:** capacity, time-windows і reachability перевіряються в одному циклі.

11.4. Чому elitism = 2

Елітизм запобігає втраті найкращого розв'язку через стохастичку операторів. Розмір 2 (4% популяції) — компроміс: достатньо, щоб захистити best-so-far, але не настільки великий, щоб задушити різноманіття.

11.5. Чому штраф $P \cdot |U|$ замість жорсткого обмеження

У ранніх поколіннях деякі хромосоми неминуче будуть infeasible — флот, наприклад, тимчасово перевантажений. Жорстка відмова таких розв'язків (фітнес = $+\infty$) знищила б усю популяцію. Штраф $P \cdot |U|$ робить infeasible-розв'язки *дуже поганими*, але порівнянними між собою — ГА може поступово рухатися до feasible-області.

12. Підсумкова схема алгоритму

```
solve_vrp(stops, depot, fleet, graphs) |
```

▼

```
Phase 1: _assign_stops_to_modes() |  
• Dijkstra reachability per mode |  
• Greedy capacity-balanced split |
```

▼ ▼ ▼

```
drive_pool bike_pool walk_pool
```

```
| | |  
▼ ▼ ▼
```

```

Phase 2: _genetic_algorithm() x3 |
├── Init pop = NN seed + random | |
│   ▼ | |
│   └── Evaluate (Split + F) | | |
│       │ | |
│       ▼ | | |
│       Elitism (top-2) | | |
│       ▼ | | |
│       Tournament select | | |
│       ▼ | | |
│       Order Crossover (p=0.85) | | |
│       ▼ | | |
│       Swap mutation (p=0.15) | | |
│       ▼ | | |
│       New population ─ | |
│       (repeat x 150 generations) | |
└── |
    ▼
    └── _build_ga_routes() |
        ├── Reconstruct OSM full path |
        └── Compute time + distance |
    ▼
    list[VehicleRoute] → Folium visualization

```

Цей документ описує алгоритм у тому вигляді, у якому він реалізований у файлі `vrp_solver.py` на момент генерації PDF. Для перегляду актуального коду див. `_genetic_algorithm()`, `_decode_chromosome()` та `_evaluate()`.